

When Is an Agent Task Complete? A Fault-Injection Study of Evidence-Based Assessment

Dan Itkis

AgentAction.dev

Research draft - September 24, 2026

Abstract

An authorized tool invocation, a successful transport response, and a completed task are different claims. We study evidence authenticity, completeness, and temporal validity in AgentAction's existing outcome assessor. An offline artifact exercises its unmodified evaluator and observation verifier across three provider emulators, 20 fault families, and 1200 parameterized cases. Verified ingestion reduces false success from 900/960 for a transport-status control to 300/960, at 65.0% decisive coverage, but incomplete histories and stale or dishonest observations retain false completion claims. We then implement a research-only gate combining authenticated history closure with a challenge-bound resource-revision check. A separate HTTP service backed by SQLite exercises 155 cases across 31 families, including overlapping requests, lost responses, process crashes, and persistent retries. Within its stated trust assumptions, the combined gate has 0/70 false successes and 0/75 false failures, compared with 25/70 and 5/75 for latest-observation assessment. This comes at 48.3% coverage and 35/75 positive admission. All 10 cases that violate complete mediation or collector honesty remain false successes. These constructed-corpus measurements establish conditional mechanism behavior, not deployment reliability. The contribution is an executable diagnosis, a minimal evaluated remedy, and an explicit account of its availability costs and trust limits.

1. Introduction

An agent may finish its conversation while its requested effect remains incomplete. A deployment tool can accept a release whose replicas never become healthy. A refund request can be accepted before settlement. An export can finish with fewer rows than required. Even when the desired state is present, a duplicate effect or an approval violation may invalidate the task. A completion decision therefore depends on more than whether the last tool call returned without an error.

Outcome evaluation itself is well established. For example, tau-bench evaluates interactions against final database state [5]. Runtime systems also enforce constraints on proposed actions [1]. More recently, EvidenceNet has studied completion admission using current, source-bound observations in network operations [2]. The remaining question addressed here is narrower: what happens when an implemented completion assessor receives incomplete histories, authenticated but temporally misleading observations, or evidence presented at the wrong trust boundary?

We investigate this question through a case study of AgentAction [9]. Its local evaluator checks a frozen intent contract against decisions, execution receipts, observations, and job metadata. A separate ingestion component authenticates observations and checks their lifetime and binding. This separation provides a useful experimental boundary: it permits us to isolate the value of verification while testing what verification cannot establish.

We contribute an executable fault-injection artifact with separate state-based ground truth; a comparison of seven transparent mechanism configurations; and a minimal closure-and-revision remedy evaluated with five configurations against a persistent HTTP service. The second study adds actual transaction boundaries, overlapping requests, lost responses, and process crashes to the initial emulators. Both studies retain failures of the

evaluated mechanisms. Neither estimates real-world incident frequency nor ranks agent models. Scenario strata are the unit of fault coverage; parameter variations check consistency rather than inflate statistical confidence.

The central finding is that increasing evidence integrity does not remove the need to specify what the evidence covers. A signature can authenticate a partial history. A recent observation can precede a state change. A universal predicate can confuse historical incompleteness with current failure. These are distinguishable assumptions that should appear in a completion contract and in its evaluation protocol.

2. Related Work

Runtime enforcement. AgentSpec specifies triggers, checks, and interventions for LLM-agent actions [1]. Our experiment studies the downstream assessment of an already executed or attempted task. It does not compare enforcement effectiveness with AgentSpec, and it introduces no new runtime policy language.

Completion admission. EvidenceNet is the closest prior work [2]. It combines completion contracts, brokered observations, source binding, network epochs, deterministic conditions, and verifier assessment. Its evaluation distinguishes admission from external task truth and includes controlled evidence interventions. We therefore do not claim novelty for completion contracts, independent outcome checks, or content-only controls. Our case study examines a separate implementation, explicit unknown outcomes, absent versus empty streams, and opposing effects of historical and current-state predicates. Its service extension isolates history closure and resource-revision checking through ablations and transactional failure schedules. Revision invalidation is closely related to EvidenceNet's epoch mechanism; neither that idea nor signed history commitments is claimed as new. Our contribution is the measured interaction and reproducible limits in this setting, not superiority to EvidenceNet.

Execution provenance and composition. Proof of Execution connects governed execution to verifiable contract, effect, history, and replay evidence, subject to explicit cryptographic and deployment assumptions [3]. CONTINUITY studies authenticated security context across component boundaries and evaluates a deterministic fault suite [4]. Our provenance-misuse control examines one such boundary, but our task is assessment rather than effect authorization. Neither system is reimplemented in our experiment.

Attestation and evaluation. in-toto supplies a precedent for verifying provenance across a software supply chain [7]. RATS distinguishes evidence, appraisal policy, and attestation results and discusses freshness mechanisms [8]. We apply an analogous distinction at the task-assessment level without claiming RATS conformance. Selective classification motivates reporting coverage together with error when a method can abstain [6]. We use descriptive finite-corpus metrics and do not inherit a learned classifier's statistical guarantees.

3. Problem and Trust Boundaries

3.1. Task truth and exposed evidence

Let C describe a task's required outcomes, hard constraints, and evidence requirements. Let W be the provider state and effect history after execution. An independent oracle defines $Y(C, W)$, which is true only if the required state exists and the execution satisfies the hard constraints. The assessor does not see W directly. It sees an evidence view E , possibly incomplete or corrupted, and returns S (qualified success), F (observed failure), or U (indeterminate).

In our workflows, success requires the correct domain state, approval, and exactly one side effect. A replay returning an existing result produces no new side effect. The oracle reads complete emulator state and does not invoke the production predicate evaluator. A provider observation, by contrast, reports the latest state of a resource; it does not certify that every historical effect has been included.

The completion decision is distinct from task truth. An unsuccessful job can legitimately receive U when the available evidence cannot establish its outcome. A successful job can also receive U. Conversely, a positive decision supported by authenticated input may still be false if the authenticated input omits a relevant effect or no longer describes the current world.

3.2. Verification and interpretation

We distinguish four assumptions. First, the contract authority defines the intended task correctly. Second, an ingestion boundary establishes an observation's issuer, signature, lifetime, and task binding. Third, evidence producers report relevant state truthfully. Fourth, the evidence view covers the events and state changes needed by the contract. The experiment explicitly violates each relevant evidence assumption in a labeled scenario rather than treating them all as signature failures.

The production local evaluator consumes normalized records whose provenance has already been established. It checks the provenance metadata and canonical payload digest, but it does not independently verify a compact JWS or enforce the ingress freshness policy. Passing arbitrary caller-created provenance objects directly to it violates that interface assumption. We include this misuse as a diagnostic control and distinguish it from the verified-ingress configuration.

Decision and receipt streams are supplied by a trusted harness. Their transport authentication, complete mediation, and persistent storage are outside this experiment. Observation signatures use fresh RSA-2048 keys and the actual production RS256 verification path. The issuer's public key is returned by an in-process JWKS substitute; no external service is contacted. A designated trusted-signer-falsehood scenario deliberately violates producer truthfulness and is an out-of-trust-model control.

3.3. An observability limit

Proposition. Suppose two allowed worlds W^+ and W^- have opposite oracle labels but expose the same contract and evidence to a deterministic assessor. Any decisive label on that shared input is wrong for at least one world.

Proof. The assessor receives identical inputs and therefore emits the same label for both worlds. If that label is S, it is wrong for W^- . If it is F, it is wrong for W^+ . U avoids a false decisive statement. The same argument prevents a randomized assessor from guaranteeing an error-free decisive answer in both worlds. This is an elementary indistinguishability observation, not a new general impossibility theorem.

Our missing-observation pairs instantiate the proposition. One world has settled or completed and one has not; both expose identical contracts, allow decisions, executed receipts, and no outcome observation. The cases show why replacing every unknown with either success or failure cannot solve the assessment problem.

4. Implementation Under Study

AgentAction issues per-job contracts from versioned profiles. The profile fixes required predicates, constraints, and trusted-observation requirements; typed variables bind job-specific resources and desired values. Contract digests accompany evidence. The evaluator independently reports outcome status, constraint compliance, qualified success, execution discipline, and an evidence-confidence field [9].

Our profiles select provider-state observations by predicate and compare their fields with the issued variables. Refund outcomes require the target payment, expected amount, and settled status. Deployment outcomes require the target service, intended version, and healthy replica count. Export outcomes require the dataset, row count, approved destination, and redacted status. Constraints require approval on all visible decision events and at most one visible executed receipt. An idempotent replay is excluded from the execution count. The at-most-one constraint intentionally depends on history completeness; a final state alone does not prove that constraint.

The hosted observation verifier checks issuer policy, RS256 signature, audience, subject, task and tenant binding, payload digest, and lifetime. The corpus uses its existing default-compatible age limit, without introducing an epoch mechanism. Rejected observations are removed before evaluation. Their rejection creates unavailable support, not a factual assertion that the task failed.

The local evaluator distinguishes an absent source from an explicitly supplied empty array. An absent source makes its predicate indeterminate. An empty array can satisfy a count upper bound. That behavior is sensible if an empty array certifies a complete stream containing no events; it is unsafe if an exporter silently substitutes an empty array for an unavailable stream. The omission experiment makes this representation issue explicit.

The existing gateway can freeze evidence snapshots and final evaluations. Targeted existing tests cover finalization, retry idempotency, profile issuance, and profile-scoped rollups. Those tests supply architectural context; this paper's repeated local evaluations are not measurements of distributed durability or proof that a frozen snapshot is complete. Freezing preserves the evidence selected, including any omissions already present.

5. Experimental Method

5.1. Corpus and independent oracle

The original artifact contains 1200 cases: three domains, 20 scenario families, and twenty parameterizations per domain-scenario combination. Seven configurations produce 8400 primary assessments. The resulting 60 strata are the unit of fault coverage. Identifiers and domain values vary across parameterizations, while each scenario's structural fault is fixed. All three domains use independently written imperative state checks, although the workflow shapes remain deliberately simple.

The refund emulator maintains refund records with payment, amount, settlement, and approval fields. The deployment emulator maintains release records with service, version, healthy replica count, and approval. The export emulator maintains artifact records with dataset, rows, destination, redaction, and approval. Each oracle checks the complete corresponding history, including its cardinality. Evidence generation reads selected state and records, then applies the labeled intervention. Assessment functions receive a projected input with no world state, scenario name, or oracle label.

The corpus includes 240 oracle-positive and 960 oracle-negative cases. This class mix is designed for diagnosis and is not intended to resemble deployment prevalence. There are 60 cases per scenario family. Exact counts are reported without confidence intervals over the parameterizations because these variants are not independent samples of operational failures. Results are also retained separately by domain and scenario.

5.2. Configurations

Tool maps the final HTTP status to success or failure. **Trace** requires a visible executed receipt and only allow decisions; it abstains if either stream is empty or absent. These intentionally weak controls isolate what can be inferred from transport and execution signals alone. They do not represent the maximum capability of a tracing product.

Content-any applies the production task predicates after a research adapter normalizes away provenance, digest, and binding failures. It is an explicitly insecure mechanism ablation, not a recommended ingestion path.

Local-any calls the unchanged local evaluator on records claiming verified provenance. **Ingress-any** first invokes the unchanged observation verifier and then the same evaluator.

Ingress-all uses the same components with universal observation predicates. It receives a separately issued profile and correctly rebound and re-signed evidence; an issued contract is never edited. **Ingress-latest** uses a research-only adapter to select the most recent verified observation before evaluating it. A tied latest timestamp with conflicting values causes abstention. This adapter is not shipped production behavior and assumes the signed

observation time is trustworthy.

The outcome observations in this corpus contain all required state fields together, so selecting a latest record is unambiguous. The adapter does not implement distributed version reconciliation, partial-field merging, or a general solution for multiple independent observers. No configuration is advertised as a reproduction of a named prior system.

5.3. Metrics and repeated execution

We map the production receipt to S when qualified success is true; to F when any required outcome or hard constraint fails; and to U otherwise. Thus a partial outcome with only missing support is U, while an explicit violated constraint can justify F despite a missing outcome. We preserve the original receipt dimensions in the saved assessment records.

For each method, N^+ and N^- denote oracle-positive and oracle-negative cases. False-success rate is the number of S labels on negative cases divided by N^- . False-failure rate is the number of F labels on positive cases divided by N^+ . Coverage is (number of S plus number of F)/N. Selective error is the number of incorrect decisive labels divided by the number of decisive labels. Positive admission is S on positive cases divided by N^+ . Undefined rates are represented as null rather than zero.

We ran the entire correctness artifact twice with independently generated keys. Saved cases, normalized assessment outputs, and summaries match byte-for-byte. Source hashes identify the tested production modules and experiment code. Persisted evidence does not include signing keys or JWS envelopes; reproducing cryptographic verification requires rerunning the artifact with fresh keys. The saved data are research observations, not third-party cryptographic attestations of the experiment itself.

5.4. Sensitivity and timing

An observation-loss sweep removes observations for a nested subset of parameterizations, identically across clean and accepted-pending cases. Its five loss levels are zero, one quarter, one half, three quarters, and all observations. Each level has 120 cases balanced by oracle label. A separate omission sweep exhausts all eight subsets of the three evidence streams, using both absent and empty representations, over clean and duplicate-visible cases. This yields 1920 assessments with Ingress-any.

The microbenchmark runs a fixed clean refund case after warm-up, using five sequential blocks per method. Signed envelopes are prepared before timing. Ingress timing includes the production verifier, a local JWKS response, JSON decoding, key import, and evaluation; it excludes signing, network, provider execution, model inference, and durable storage. Hardware and raw samples are saved. Fixed method order and lack of CPU isolation limit fine-grained comparisons.

6. Results

6.1. Aggregate behavior

Table 1 reports the complete corpus, including the explicitly out-of-model trusted-signer control. These are fault-suite outcomes, not security guarantees. Ingress-any makes 300 false success claims, compared with 900 for Tool and 600 for Content-any. Its coverage falls to 65.0% because invalid or missing observations often warrant abstention. It admits 180 of 240 genuinely successful cases; the missing-success family accounts for the remainder.

Method	FS/N-	FF/N+	Coverage	Sel. error	Positive admit
Tool	900/960	0/240	100.0%	75.0%	240/240
Trace	840/960	0/240	95.0%	73.7%	240/240
Content-any	600/960	0/240	90.0%	55.6%	180/240
Local-any	420/960	0/240	75.0%	46.7%	180/240
Ingress-any	300/960	0/240	65.0%	38.5%	180/240
Ingress-all	240/960	60/240	65.0%	38.5%	120/240
Ingress-latest	240/960	0/240	65.0%	30.8%	180/240

Table 1. Exact results on 1200 cases. FS/N- and FF/N+ show false-success and false-failure counts with their class denominators. Coverage and selective error expose the cost of abstention. All methods see matched task and evidence content; the universal profile changes observation quantification explicitly.

The verification boundary matters. Local-any rejects the wrong issuer, invalid payload digest, and wrong task binding, but it accepts forged provenance metadata with an invalid signature and observations whose envelope has expired. Ingress-any abstains on those additional families after actual verification. This is evidence for correct composition of the existing components, not evidence that local metadata fields can replace authentication.

Figure 1 exposes every scenario's outcome. All parameterizations and domains produce the same categorical pattern within each scenario-method cell. That consistency is useful as a conformance check and also reveals the corpus's limited behavioral diversity.

6.2. Valid signatures do not close all evidence gaps

Five families remain false successes under Ingress-any: conflicting observations, a state regression within the allowed age window, a hidden duplicate receipt, an empty receipt stream, and a lying trusted issuer. Each contributes 60 false successes. The latter is outside the truthful-producer assumption; the history-omission cases violate completeness, and the temporal cases test how task semantics are expressed.

In the hidden-duplicate case, two effects occur but only one executed receipt is exposed. The provider observation reports the latest resource state, which is correct in isolation. In the empty-stream case, both receipts are omitted and represented as an empty list. The count upper bound passes in both cases. Authentication of the state observation cannot establish the cardinality of a separate history.

The within-TTL regression is also revealing. A truthful successful observation is collected before the world changes. Its signature and age remain valid when evaluated, but its statement no longer establishes the current outcome. Clock-based freshness bounds observation age; it does not prove the absence of an intervening state transition. The epoch approach in EvidenceNet addresses a different freshness assumption [2]. We do not implement that approach here.

	Tool	Trace	Content	Local	Ingress any	Ingress all	Ingress latest
clean (+)	S	S	S	S	S	S	S
explicit failure (-)	F	F	F	F	F	F	F
accepted pending (-)	S	S	F	F	F	F	F
wrong target (-)	S	S	F	F	F	F	F
unapproved action (-)	S	S	F	F	F	F	F
duplicate visible (-)	S	S	F	F	F	F	F
idempotent replay (+)	S	S	S	S	S	S	S
missing success (+)	S	S	U	U	U	U	U
missing failure (-)	S	S	U	U	U	U	U
untrusted issuer (-)	S	S	S	U	U	U	U
payload tamper (-)	S	S	S	U	U	U	U
binding mismatch (-)	S	S	S	U	U	U	U
expired (-)	S	S	S	S	U	U	U
forged provenance (-)	S	S	S	S	U	U	U
conflict (-)	S	S	S	S	S	F	F
within ttl regression (-)	S	S	S	S	S	S	S
hidden duplicate (-)	S	S	S	S	S	S	S
empty receipt stream (-)	S	U	S	S	S	S	S
trusted signer falsehood (-)	S	S	S	S	S	S	S
legitimate state evolution (+)	S	S	S	S	S	F	S

(+) true success; (-) true failure. Red = incorrect decisive assessment.

Figure 1. Scenario-level assessment matrix. S, F, and U indicate success, failure, and abstention. Red S/F cells disagree with oracle truth; gray cells abstain. Each cell summarizes 60 structurally related cases, not independent trials.

6.3. Quantifier and temporal-policy tradeoffs

For a success observation followed by an observed regression, existential predicates accept the earlier positive evidence. Universal predicates reject this mixture. However, universal predicates also reject a legitimate pending-to-success transition because the earlier negative record remains in the evidence set. Ingress-all therefore exchanges 60 false success claims for 60 false failure claims in the corpus; its selective error is unchanged from Ingress-any.

Ingress-latest resolves both represented transitions correctly, reducing false success to 240 while retaining zero false failures in this corpus. It still accepts the within-TTL regression when no later observation exists, omitted histories, and the trusted falsehood. The result supports temporal policy selection for a narrow state model; it does not establish latest-write-wins as a generally sound completion rule.

6.4. Observation loss and empty streams

The loss sweep reveals an unavoidable information tradeoff. With complete outcome observations, Ingress-any separates clean and accepted-pending states. As observations are removed, its coverage falls linearly to zero while it makes no false decisive claims in this balanced subexperiment. Tool remains fully decisive with 50.0% selective error because the same successful transport response accompanies both labels. At total observation loss, each positive/negative pair exposes the same assessment input, as in Section 3.3.

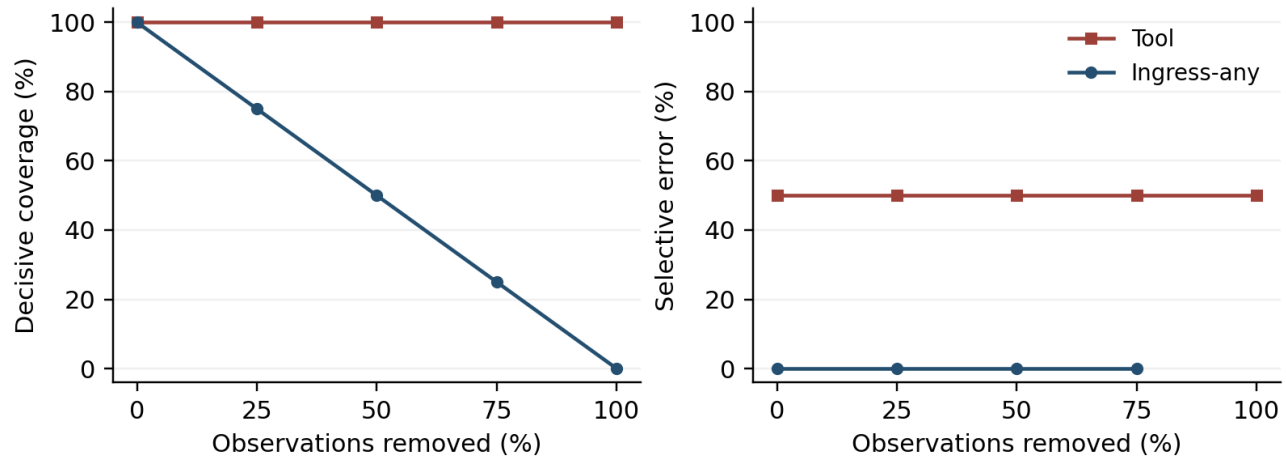


Figure 2. Coverage and selective error as paired outcome observations are removed. Selective error is undefined at zero coverage and is not plotted there. The sweep characterizes designed information loss, not a natural missing-data distribution.

The stream sweep shows a distinct representation hazard. When only the receipt source is absent, Ingress-any abstains on all 120 clean/duplicate cases. When the same source is supplied as an empty array, it instead returns success on all of them, including 60 duplicate cases. The result is specific to a count upper bound and an exporter that does not certify completeness. It does not imply that every empty array should be rejected.

Receipt source	False success	Abstentions	Coverage
Absent	0/60	120/120	0.0%
Empty	60/60	0/120	100.0%

Table 2. Receipt-only omission slice of the exhaustive stream sweep. Both representations expose the same remaining decisions and observations. The complete mask matrix is released with the artifact.

6.5. Evidence confidence and local cost

Every false success from Ingress-any has evidence-confidence equal to one. The implementation's field counts determinate predicates and present requirements; it is not a probability that the conclusion is correct. This is consistent with its arithmetic definition but makes calibration language inappropriate. Any dashboard using the field should preserve that distinction.

The measured machine was Apple M5 Pro running darwin arm64 and Node v23.11.0. Table 3 reports local component latency and the actual count of timing samples. These measurements establish that the small fixed workload can be evaluated locally at modest cost. They do not estimate end-to-end agent latency or compare fairly with published performance numbers on other hardware and workloads.

Method	Median (ms)	95th percentile (ms)
Tool	0.0001	0.0010
Trace	0.0002	0.0003
Content-any	0.0541	0.1103
Local-any	0.0401	0.0728
Ingress-any	0.1273	0.2603
Ingress-all	0.1153	0.1938
Ingress-latest	0.1162	0.1771

Table 3. Local fixed-case microbenchmark in milliseconds. Each method has 1000 measured calls after warm-up. The very small Tool and Trace measurements include timer and loop overhead. Differences among the three ingress variants should not be interpreted as stable performance rankings. Raw blocks, input byte counts, and envelope sizes are available in the artifact.

7. A Conditional Remedy with Persistent-Service Validation

7.1. History closure and a final revision check

The surviving errors motivate two explicit prerequisites for a completion decision. First, the received history must be complete for the state observation's boundary. Second, the observation must still describe the resource at the chosen assessment point. We implement these prerequisites as a research-only wrapper around the same production verifier and evaluator. No production package or gateway behavior changes.

The trusted broker obtains the resource state, its revision r , and all decision/receipt records from one service snapshot. It constructs a closure checkpoint containing r , each stream's count, and a SHA-256 digest of the complete records sorted by their sequence number. The checkpoint is included in the signed observation payload, binding it to the contract, task, resource, and observed values. Closure validation requires both streams, exactly the committed counts, contiguous sequence numbers starting at one, and matching content digests. Arrival order may vary; missing records, repeated sequence positions, substituted content, and a truncated suffix fail validation. A supplied empty stream passes only when the authenticated checkpoint also commits to zero records.

For temporal validation, the trusted collector sends a fresh random challenge to the service after evidence collection and retains the expected value independently of the response. The broker authenticates the returned task, resource, challenge, and current revision h in a separate observation envelope. The gate requires a valid signature and binding, the retained challenge, and $r = h$. An old signed response with a different challenge cannot satisfy this check. The low-level research validator consumes trusted challenge context; allowing an evidence submitter to choose both proof and expected challenge would invalidate the freshness premise. Wall-clock lifetime checks still run, but do not substitute for revision equality. The research logical clock is fixed for reproducibility; this extension tests revision freshness, not deployed clock synchronization.

All three remedy configurations use the same latest-observation selection as Ingress-latest. Closure enables only history validation; Revision enables only the final revision check; Closure + revision enables both. Each requires the checkpoint's revision metadata. Any enabled prerequisite that is missing, invalid, or inconsistent yields U before the outcome predicates run. Otherwise, the unchanged evaluator determines S, F, or U from the selected current-state observation and supplied histories. This conservative ordering may abstain even when part of the input contains a violation. It is a deliberate availability cost, not a reassignment of U to failure.

Conditional justification. Assume the service and broker are honest, the checkpoint represents the full relevant history, relevant state/history changes atomically advance a persistent monotonic revision, the signatures and digests hold, and the contract's predicates correctly encode task truth. Matching closure commitments establish that the evaluated histories equal the histories at r . An authenticated, challenge-bound final read with $h = r$ establishes that no relevant committed change separates that snapshot from the final read. A decisive evaluation therefore refers to the intended history and state at that point. This is a model-relative argument, not a general verification theorem: omitted effects, counter rollback, incomplete predicates, or a dishonest collector break its premises. A later change can invalidate the state after the final read; the gate neither locks the resource nor atomically authorizes a downstream action.

7.2. Service, schedules, and independent truth

The validation service runs in a separate Python process and accepts actual loopback HTTP requests. Its SQLite database persists jobs, refund effects, idempotency keys, decision/receipt events, and per-job revisions. Ordinary mutations use one write transaction for the effect, event, and revision update. A replay records a new receipt but reuses the existing effect. Evidence reads use one read transaction. SQLite WAL mode permits a reader to see an earlier committed snapshot while a writer has uncommitted changes [11]. The service uses synchronous FULL mode; the experiment tests process crashes, not power-loss behavior.

Explicit barriers establish concurrency. For overlapping writes, the first request pauses before commit and the second request's arrival is confirmed before releasing the first. For the stale-snapshot schedule, the reader returns while the duplicate-effect writer remains blocked; that writer then commits before the final revision read. Other cases close the TCP connection after commit and retry the same key; kill and restart the process after commit; or kill it while a second effect is uncommitted. The latter checks that state, ledger, and revision roll back together. These schedules exercise actual service/storage boundaries, although their ordering is controlled rather than sampled from operational traffic.

A separate read-only Python auditor queries the database directly after the final head read, while writers are quiescent. Truth requires exactly one correct settled refund and approval of every logged request, including retries, matching the contract's universal approval policy. The audit's revision must match the final head revision. It does not call the service's evidence endpoint or the production evaluator. The assessor receives only its contract, exposed evidence, challenge, and head proof; raw tables, oracle labels, and scenario names are excluded. A second Python reference implementation checks the saved finite-domain decisions and recounts every metric. Cryptographic verification itself is rerun through the production JWS implementation. These implementations are separate code written for this project, not an external organization's independent evaluation.

The service corpus has 31 scenario families with five amount/identifier variations each: 155 cases and 775 assessments across five methods. It includes benign progression, regression, wrong resources, unapproved and duplicate effects, stream omissions and substitutions, missing metadata, signature faults, challenge replay, and the concurrency/restart schedules above. 145 cases preserve the service trust assumptions; 10 controls intentionally violate complete mediation or collector honesty. Both strata are retained. The same evidence is supplied to each method; only enabled safeguards differ. Variations are repeated structural checks, not independent deployment samples.

7.3. Results and availability cost

Method	FS/N-	FF/N+	Coverage	Unknown/N	Positive admit
Ingress-any	30/70	5/75	89.7%	15/145	65/75
Ingress-latest	25/70	5/75	89.7%	15/145	65/75
Closure	10/70	5/75	69.0%	45/145	50/75
Revision	15/70	0/75	65.5%	50/145	45/75
Closure + revision	0/70	0/75	48.3%	75/145	35/75

Table 4. Persistent-service results within the stated service/broker assumptions: 145 cases, including 75 positive and 70 negative cases per method. The two excluded trust-violation families are reported explicitly below, not removed from the released corpus. Closure and Revision are ablations of the combined research gate.

Closure eliminates the hidden-duplicate, empty-receipt, and hidden-unapproved-decision false successes, but still accepts stale success and the snapshot preceding a duplicate commit. It also rejects a stale failing observation after the task has become successful. Revision prevents those temporal mistakes, but accepts incomplete histories at the current revision. Their combination produces 0 false successes and 0 false failures in this stratum, while preserving successful admission for normal completion, lost-response retries, process restarts, crash rollback, concurrent idempotent requests, reordered records, and observed progression.

The combination's decisive coverage is only 48.3%: 75/145 inputs are U, and only 35/75 true successes are admitted. Figure 3 makes that loss of availability visible. Zero observed error alone would be uninformative for an always-abstaining method; this gate makes 70 correct decisive assessments, but still declines many true successes when their evidence is unavailable or invalid. A single read-only recollection could plausibly restore coverage for transient omissions, yet this study does not measure a repair policy or assume that recollection succeeds under continuing writes.

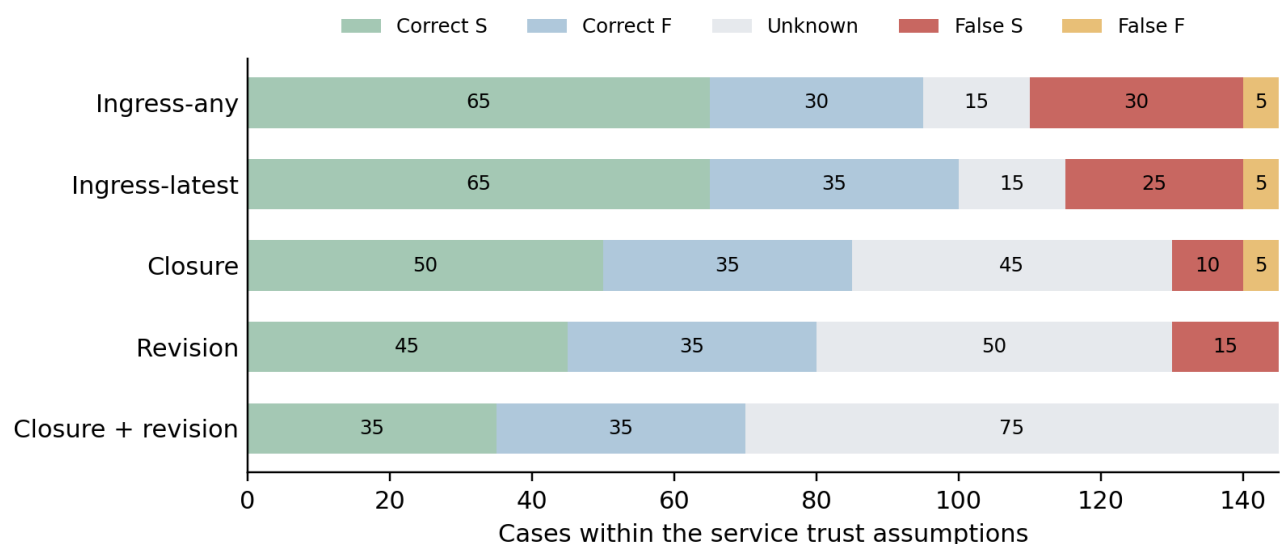


Figure 3. Counts of correct, unknown, and incorrect service decisions within the trust assumptions. Each bar contains the same 145 cases. Counts reflect the deliberately selected fault mix; they are not estimated production frequencies.

The unmediated-effect control adds a refund without updating the ledger or revision. The dishonest-collector control signs a settled-state claim when the stored refund remains pending. Every method, including the combined

gate, incorrectly admits all 10 such cases. Across the complete service corpus, the combined gate therefore has 10/80 false successes, zero false failures, and 51.6% coverage. Authentication and closure validate a trusted reporting path; they cannot establish that unreported activity or a lying authority does not exist. A separate boundary test also confirms that an earlier valid completion claim does not remain valid after a later effect; obtaining a new head invalidates its old revision.

7.4. Local service cost

Method	Gate p50 (ms)	Gate p95 (ms)	HTTP p50 (ms)	HTTP p95 (ms)
Ingress-any	0.1350	0.2227	0.9495	1.7688
Ingress-latest	0.1354	0.1979	0.9439	1.7430
Closure	0.1408	0.2122	0.9525	1.6807
Revision	0.2168	0.3100	1.7983	2.7869
Closure + revision	0.2198	0.3465	1.7983	2.9628

Table 5. Milliseconds per clean service assessment. Each entry uses 200 samples after 20 warm-ups, in five blocks with rotating method order. Gate measurements use prepared envelopes and exclude signing/HTTP. HTTP measurements include a fresh SQLite-backed snapshot read, broker construction and signing, and gate evaluation; revision methods also fetch and authenticate a fresh head. Action execution, process startup, remote JWKS/TLS, and model inference are excluded.

The service timing environment is Apple M5 Pro, darwin arm64, Node.js v23.11.0, Python 3.13.2, and SQLite 3.49.1. The additional head request and signature verification increase local cost; the measurements do not establish performance on remote providers, larger histories, contended databases, or multiple resources. History verification hashes all supplied records, so its work grows with history size. The fixed small task, lack of CPU isolation, and local transport limit the interpretation of the measured percentiles.

8. Implications for Completion Contracts

Specify evidence completeness separately from presence. A source being supplied is not equivalent to a complete account of that source. A stronger contract could require a provider-issued range, sequence closure, a count commitment, or a reconciled state query. Such mechanisms must state which effects can bypass the reporting path. Merely adding another field or signature does not prove coverage.

Specify temporal scope. A predicate can concern a past event, current state, or an interval invariant. Those meanings require different aggregation. Existential evaluation is reasonable for evidence that an event happened at least once. A current-state claim needs a valid observation after relevant changes. An interval invariant needs coverage of the interval; neither selecting the latest sample nor requiring all received samples to pass proves that coverage.

Preserve abstention. Missing evidence should remain visible in both individual assessments and aggregate reporting. An unknown case should not disappear from the denominator, be labeled a business failure, or be counted as successful simply because no violation was recorded. The loss experiment provides matched examples for all three mistakes.

Keep verification at the boundary. The distinction between Local-any and Ingress-any demonstrates a composition obligation. Caller-provided provenance metadata cannot authorize itself. Conversely, an authenticated issuer's statement should not be treated as an independently established fact when the issuer's collection process or truthfulness lies outside the trust assumptions.

Freeze definitions and evidence without overstating what freezing proves. Versioned profiles make like-for-like assessment possible, and a frozen snapshot makes a particular assessment repeatable. Neither repairs a poorly specified goal nor supplies missing events. The artifact isolates these properties instead of treating one signed final score as evidence of all of them.

9. Limitations and Threats to Validity

This is an implementation case study comprising deterministic in-memory emulators and an author-built persistent HTTP/SQLite service. The extension exercises real sockets, transactions, overlapping requests, and process crashes, but contains no customer workloads, LLM-generated trajectories, commercial provider sandbox transactions, multi-host races, or independently authored benchmark tasks. Its domains share a simple one-effect goal. The service adds operational mechanisms, not external task diversity. Parameter variations check consistency across identifiers and values, not broad generalization; results should be interpreted at the scenario-family level.

The same project produced the implementation, profiles, scenarios, and research artifact. The oracle is separated in code and reads complete state, but it is not an independent organization's adjudication. The harness and oracle may share conceptual mistakes. Tests check projected input isolation and retain counterexamples, but external reproduction and externally authored tasks would provide stronger validation.

The corpus intentionally overrepresents failures and includes violated trust assumptions. Pooled false-success percentages depend on that mix. A different weighting changes the reported percentage without changing any mechanism. For this reason we release case-level results and the full scenario matrix and make no population-level claim. An actual deployment's permitted worlds and completeness guarantees would need separate evaluation.

The baseline methods are transparent mechanism controls, not production configurations of tracing platforms or named research systems. They were selected to isolate evidence capabilities and can be strengthened. The study therefore cannot demonstrate superiority to EvidenceNet, AgentSpec, CONTINUITY, or Proof of Execution. It also cannot establish that the listed failure families are novel individually.

Authentic signatures were checked, but key discovery was local and keys were ephemeral. Neither study measures TLS, JWKS caching, rotation, revocation, remote issuer availability, or credential compromise. The original experiment assumes decision/receipt authenticity. The service extension binds their contents through the broker's signed closure checkpoint, assuming honest service collection and complete mediation. It does not implement production durable evidence ingestion. Latest-observation selection trusts timestamps and handles only the single-observer full-state shape exercised here.

Correctness reruns are deterministic after documented challenge normalization, while timing is not. The original microbenchmark excludes network, provider, and model costs and uses fixed method order. The service timing rotates method order and adds local HTTP/storage reads, but still excludes remote providers and models. Neither isolates the CPU. Precision should not be mistaken for portability. The service gate depends on a monotonic revision across relevant updates and restarts; restoration from backups, resource incarnation changes, multiple authorities, and distributed atomic admission need additional mechanisms. All claims concern the chosen assessment point, not eventual success, causal responsibility, or permanent satisfaction.

10. Conclusion

The experiments identify distinct limits of evidence-based completion assessment. Outcome predicates improve on transport and execution signals, while verified ingestion rejects evidence that metadata checks cannot authenticate. Authentic input can nevertheless remain incomplete or temporally inadequate. A minimal closure-and-revision gate removes the observed decisive errors within a specified persistent-service trust model, at a substantial cost in abstention, and retains false successes when complete mediation or collector honesty fails. The released implementation, controlled service schedules, raw tables, and assessment records make both the remedy and its limits inspectable. Completion contracts need explicit statements of history coverage, temporal scope, and the authorities on which those statements depend.

Reproducibility, Disclosure, and Author Responsibility

The artifact is in the AgentAction repository under `research/completion-assessment` [10]. Both result manifests hash the evaluated source files, including the unchanged production evaluator and observation verifier. The protocols, generators, oracles, tests, normalized case records, service table exports, row-level decisions, summaries, timings, and manuscript builder are included. Correctness requires Node.js with TypeScript stripping support and, for the service extension, Python with standard-library SQLite. The service binds only to loopback and uses temporary databases; no external provider is contacted. Correctness reruns generate fresh signing keys and challenges. Saved challenges use case-specific placeholders with recomputed head digests, enabling byte comparison but not preserving independently verifiable signed attestations. Manuscript rendering uses pinned Python dependencies.

The author is affiliated with AgentAction.dev and has a direct interest in the evaluated project. AI-assisted tooling was used to inspect code and literature, implement the experimental harness, analyze outputs, and prepare this draft. It is not listed as an author. This draft has not undergone independent peer review. All evaluated records are synthetic and no customer data or production credentials are included.

References

- [1] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. **AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents**. arXiv:2503.18666v3; accepted at ICSE 2026, 2025. [Source](#)
- [2] Tianzhu Zhang, Chih-Kai Huang, and Meikang Qiu. **Can AI Agents Deliver Verifiable Network-Wide Outcomes Across Authority Boundaries?**. arXiv:2609.10181v2, 2026. [Source](#)
- [3] James Rhodes and George Kang. **Proof of Execution: Runtime Verification for Governed AI Agent Actions**. arXiv:2607.05397v1, 2026. [Source](#)
- [4] Chris Zheng and Geng Yang. **CONTINUITY: Security-Context Contracts for Composable LLM Agent Controls**. arXiv:2609.05269v1, 2026. [Source](#)
- [5] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. **tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains**. arXiv:2406.12045v1, 2024. [Source](#)
- [6] Yonatan Geifman and Ran El-Yaniv. **Selective Classification for Deep Neural Networks**. arXiv:1705.08500v2, 2017. [Source](#)
- [7] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. **in-toto: Providing farm-to-table guarantees for bits and bytes**. 28th USENIX Security Symposium, pp. 1393-1410, 2019. [Source](#)
- [8] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. **Remote ATtestation procedureS (RATS) Architecture**. RFC 9334, 2023. [Source](#)
- [9] AgentAction contributors. **AgentAction: intent evaluator and observation verifier**. Source revision 856c3739f18a7d6cda775ab187f865278dc6e759, 2026. [Source](#)
- [10] Dan Itkis. **Completion-assessment research artifact**. AgentAction; emulator and service protocols with source and data hashes in their result manifests, 2026. [Source](#)
- [11] SQLite project. **Isolation In SQLite**. Official documentation, updated April 18, 2022, 2022. [Source](#)